

Evaluation of NIST Lightweight Cryptography Algorithms for Industrial Internet of Things

Hussein A. Al-shmailawi¹, Emad H. Al-Hemiary², Axel Sikora³

¹Department of Information and Communications Engineering, College of Information Engineering Al-Nahrain University, Baghdad, Iraq, hussein.ali@coie-nahrain.edu.iq

²Department of Networks Engineering College of Information Engineering, Al-Nahrain University Baghdad, Iraq, emad@coie-nahrain.edu.iq

³Institute of Reliable Embedded Systems and Communication Electronics Offenburg University Offenburg, Germany, axel.sikora@hs-offenburg.de

Abstract—Lightweight cryptography (LWC) has gained abundant attention lately, especially when discussing security in Industrial Internet of Things (IIoT) applications and other systems with constrained resources. IIoT has raised various security issues, highlighting the necessity for protection mechanisms. Traditional cryptographic algorithms, such as AES, require significant memory and processing power. LWC, on the other hand, promises security solutions that can meet the needs of IIoT devices with constrained resources. This paper systematically investigates how well the selected algorithms from the NIST CAESAR competition (Ascon, TinyJambu, Schwaeam, Xoodyak, Elephant, Photon-Beetle, and Grain) perform in deeply embedded systems, such as ESP32 microcontrollers.

The main contributions of this study include a comprehensive evaluation, including throughput, performance on various data sizes, an initialization time of LWC algorithms, memory usage, CPU utilization, and randomness analysis, along with the calculation of a target function based on critical metrics in IIoT environments. The findings were that ASCON outperformed TinyJambu regarding cryptographic statistical analysis and performance at the expense of highly required memory. When data sizes are less than 64 bytes, TinyJambu, similar to Ascon, offers outstanding throughput and efficiency in memory utilization compared to other LWCs. When comparing LWC results to AES, the throughput, latency, and other metrics of Ascon, TinyJambu, Schwweam, Gift, and Xoodyak were significantly higher. The outcomes of LWC showed encouraging performance, highlighting the lightweight cryptographic algorithm's potential for use in upcoming IIoT applications.

Keywords—lightweight cryptography, NIST, IIoT, Constrained devices, ESP32

I. INTRODUCTION

In the world around us, the rate at which smart devices connect to the internet is increasing rapidly. According to research, over 29 billion Internet of Things (IoT) devices will be in use by 2030, more than doubling from the 9.7 billion that already exist [1]. With over 5 billion consumer devices, the digital economy will have the most significant number of IoT devices by 2030. This growth makes the

Internet of Things an economically feasible new market that may support the expanding digital economy [2].

Table 1 compiles the attacks and security mechanisms relevant to each Internet of Things (IoT) structure level. The resource constraints of IoT systems present numerous challenges to providing security mechanisms, and traditional algorithms are resource-intensive; this constraint restricts any integration process that protects the Industrial Internet of Things [3]. This article evaluates the performance of lightweight cryptography specific to the intermediary layer and highlights the security requirements addressed by lightweight algorithms. Numerous security requirements justify the use of LWC in IoT systems.

TABLE I. ATTACKS ON LAYERS [4]

Layers	known attacks	Security objectives
Application	Dos, DDoS, malicious code injection, cross-site scripting, parameter tampering	Authentication, Privacy protection, data security management
Transport network	Sniffing, eavesdropping, routing attacks, MITM, Hello flooding, Black hole	Encryption, data integrity, data authentication, key management
Perception	Node compromised, Node capture, fake node, side channel attacks	Hash function, authentication

Traditional cryptographic techniques cannot fulfill the unique requirements of limited-resource IoT devices as they have been optimized primarily for high-performance computer applications [5] [6]. The National Institute of Standards and Technology (NIST) launched a global competition to close this gap by defining security mechanisms that could meet the strict requirements for power efficiency, battery longevity, and the limited on-chip resources essential for IoT devices to provide valuable services. LWC is the term coined to describe these specialized cryptographic algorithms [7]. Fifty-six new algorithms participated in this competition, which unfolded over three rounds. In February 2023, NIST announced the 10 finalists as the most effective standard lightweight

solutions to address various application requirements [8]. There are three categories of LWC architecture: block ciphers (GIFT-COFB [9], ISAP [10], PHOTON-Beetle [11], SCHWAEMM [12], Romulus [13], Xoodoo [14]), permutation (ASCON [15], TinyJambu [16], Elephant [17]), and stream ciphers (Grain [18]). Block ciphers and permutation ciphers encrypt a message by dividing the message into blocks and encrypting them, whereas stream ciphers encrypt a message sequentially, one bit or byte at a time. Block ciphers perform key scheduling, and the encryption process consistently employs the block cipher mode of operation. Conversely, permutation encryption enhances the capacity and speed of processing plaintext. The internal state, which is the core of permutations, is continually updated and encrypted during the permutation process, and key scheduling is unnecessary [19].

Nonetheless, defining LWC is one of the field's most significant challenges. Cryptography's ongoing goal is to find a suitable balance between "lightness" of resources and a high level of protection. However, some contend that to reduce the obstacles to the adoption of encryption on limited devices, lightweight cryptography must have a single algorithm that is superior everywhere. Some academics suggest defining lightweight cryptography as targeting specific niches.

This study contributes by comprehensively evaluating LWC algorithms. It analyzes the impact of parameters such as data size and key initialization time on performance metrics, contrasts their efficiency with AES-GCM, and computes a target function that incorporates multiple performance metrics, each assigned a weight based on the specific requirements of IIoT environments.

The remainder of this paper is organized as follows: Chapter I introduces the study and defines the research problem. Chapter II reviews key prior related work and formulates the research questions. Chapter III presents the implementation of LWC algorithms on ESP32-S2 hardware, while Chapter IV evaluates their performance using benchmark metrics. Chapter V provides a detailed discussion of the findings, while Chapter VI concludes the paper. Finally, Chapter VII lists the references.

II. STATE OF THE ART

A. Related work

In the study by I. Elsadek et al. [20], the authors assess the hardware and energy efficiency of the 10 NIST LWC standardization candidates for devices with limited resources, such as IoT and UAVs. TinyJambu outperforms other algorithms, which use more energy for identical processing activities. A limitation of this study is that it focuses on energy efficiency and throughput. It may not be an exhaustive assessment of these algorithms. Energy efficiency alone classifies algorithms into three categories (most efficient, intermediate, and least efficient), ignoring throughput. T. Kitahara et al. [21] test finalist algorithms

selected by NIST for standards on the Cortex-M0 microcontroller. The results show that the finalists use fewer resources than the AES-GCM, with Ascon and Grain optimized for performance improvements. The paper's weakness is that it is based on algorithm optimization implementations, which may reduce the latency of the encryption process but increase power consumption, resulting in impractical deployments, and does not balance this with the memory footprint, which may result in excessive RAM usage in constrained devices. In the study by C. Rahul et al. [22], the authors focus on context switches, page faults, CPU utilization, and memory usage; this research investigates LWC algorithms to improve safety and performance in resource-constrained IoT devices. The study evaluates major algorithms, such as Gift, TinyJambu, and Ascon, to connect IoT security discussions and help choose cryptographic solutions that meet the requirements of IoT devices. M. El-hajj et al. [23] analyze the software implementation of NIST Lightweight Cryptography algorithm finalists, emphasizing RAM-optimized embedded device implementations that use ARM Cortex-M3 and AVR ATmega microcontrollers. TinyJambu-128 is especially well-suited for embedded systems with constrained resources, as its implementation demonstrates better RAM efficiency compared to other NIST finalists. According to the study, improving secure communication in IoT devices requires optimizing cryptography primitives for RAM utilization. One of the paper's shortcomings involves conducting experiments with the Spike simulator. Practical hardware factors like memory and latency are not considered.

Notwithstanding the significant contributions to the performance analysis of the LWC for IoT applications, as demonstrated in the literature, particularly in the areas of the finalist of the NIST competition, there are still significant deficiencies that require attention. Initially, it is essential to use a realistic and suitable environment where the devices involved are commonly used for developing IIoT applications, allowing for effective analysis of these algorithms. Most research has focused on utilizing FPGA. Moreover, most research concentrates on hardware models, and the difficulties of realistic implementation are frequently disregarded, particularly in IIoT devices with limited resources.

B. Research Question

This research examines metrics such as the algorithm's throughput, cross-variable data size, encryption time, initialization time, memory usage, and statistical randomness test. It aims to ascertain the suitability of a specific LWC algorithm for deployment on IIoT devices with limited resources. The following main questions guide the research:

1) *Performance Comparison: How effectively can lightweight cryptography algorithms perform on tiny IIoT*

devices compared to traditional AES-like algorithms using a target function that combines key performance metrics?

2) *Resource Efficiency*: In IIoT environments, which LWC algorithms provide the best trade-off between high throughput, low CPU cycles per byte and minimal memory usage?

3) *Initialization Efficiency*: To identify LWC algorithms that minimize initialization overhead and reduce per-byte encryption time, thereby improving performance in resource-limited IIoT environments.?

Answering the above questions is essential during this study, which investigates the evaluation of lightweight algorithms.

III. IMPLEMENTATION OF LIGHTWEIGHT CRYPTOGRAPHY ON ESP32-S2

A. Development Environment Setup

The performance of various lightweight cryptographic algorithms and AES GCM mode algorithms was assessed on the ESP32-S2-WROVER Module. ESP32 is a series of low-power, cost-effective, on-chip microcontrollers that are single-threaded with integrated Wi-Fi and Bluetooth. ESP32 was created and developed by Espressif Systems. Setting up Visual Studio Code and integrating it with the Espressif IoT development framework (ESP-IDF), which includes all the necessary APIs, scripts, and an IDE for developing programs in the ANSI C standard using the gcc compiler, is essential for creating apps for the ESP32. The connections between each of the stated devices are illustrated in Figure 1. It represents the system architecture and component connections used in the project. The program is then created using the IDF command line or inside Visual Studio Code. The ESP32 will receive the application via USB, and the ESP-IDF will execute and show the results.

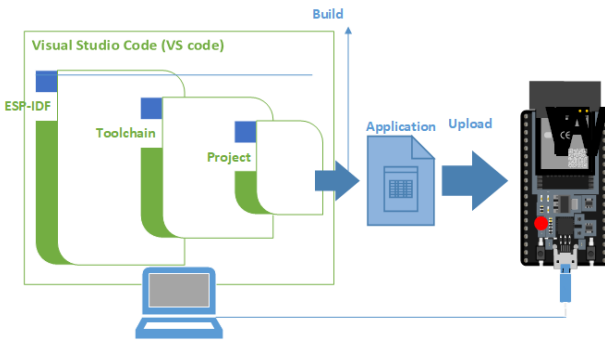


Figure 1. Development of the program on ESP32 devices

B. Implementation of the Evaluation Program

Since some lightweight encryption algorithms are still in the optimization stage and have not yet reached their final stages, developing specific functions to measure their performance is essential. The implementation involves several key steps.

1) *Assign the plaintext data to be encrypted, the private key used, the associated data, and the nonce data, ensuring they are fixed in size and consistent across all algorithms.*

2) *Utilize a high-precision time function for obtaining high-precision time measurement, as encryption and other activities are quantified in microseconds, necessitating exact temporal accuracy.*

3) *One or many loop routines to produce additional data samples.*

The evaluation program must accurately implement the algorithms. To ensure that the measurement metrics gathered by the algorithm are precise and trustworthy, it must be able to encrypt the plaintext and then decrypt the ciphertext back to the original data.

C. The contributions of this study:

1) *Comprehensive Evaluation*: According to the literature review of relevant research, no particular study has thoroughly examined lightweight algorithms with different metrics to the best of the author's knowledge. Performance and execution time are the main subjects of most studies.

2) *Initial Parameter Analysis*: In previous analyses, initial parameters such as block size or data type are set to create an appropriate environment. Some algorithms may produce different results depending on these initial parameters. However, this study examines several performance measures, such as evaluating key initialization time and measuring throughput with variable data sizes.

3) *LWC and AES Comparison*: Previous research has not thoroughly compared AES and lightweight cryptographic algorithms. This study fills this gap by providing a thorough performance analysis and comparing it to AES operating in GCM mode.

IV. RESULTS AND EVALUATION

This chapter presents a performance analysis of many aspects by measuring the key performance metrics of lightweight cryptography algorithms and AES-GCM. It quantifies the efficiency gains of LWC in resource-constrained environments, where it may outperform conventional cryptographic algorithms. Evaluating the outcomes offers insights into the efficacy and appropriateness of LWC for IIoT applications.

1) *Target Function for Evaluating LWC Performance*: To address the first RQ, we employ the target function that integrates six key performance metrics. The target is a mathematical framework that combines important metrics to assess an algorithm's performance, particularly in the context of IIoT application criteria.

$$F(A) = \sum_{i=1}^N W_i \cdot \text{NormalizedMetric}(A)_i \quad (1)$$

$$NormalizedMetric(A)_i = \frac{x_{\max i} - x_i}{x_{\max} - x_{\min} - i} \quad (2)$$

Figure 2 illustrates the results of the target function for evaluating LWC algorithms, where A denotes the evaluated LWC algorithm, W_i represents the relative weights assigned to every metric (based on IIOT application priorities), and the normalized value of $metric_i$ for the given algorithm. To compute a normalized score ranging from 0 to 1 for each metric, Equation (2) was applied. For instance, the throughput of the TinyJambu algorithm (0.6 MB/s) was normalized using the observed minimum and maximum throughput values of 0.021 MB/s and 1.76 MB/s, respectively. This yielded a normalized value of approximately 0.332, allowing for a fair comparison across all evaluated algorithms. To calculate the target function, we assigned a weight of 0.25 to CPU cycles per byte, prioritizing energy consumption in IIoT devices. Initialization time was weighted at 0.20 due to its direct impact on algorithm throughput, while throughput received a weight of 0.20 due to the importance of high-speed data processing. The remaining weights of the target function were distributed as follows: 0.15 for time per byte for real-time latency requirements, 0.15 for memory usage, and 0.05 for CPU utilization, recognizing that some algorithms may consume up to 100% of CPU resources. Results demonstrate that most of the evaluated LWC algorithms achieved higher overall scores compared to AES-GCM, with ASCON obtaining greater throughput while using fewer CPU cycles.

2) *Total initialization time*: It indicates the amount of time needed to set up the cryptographic key and the associated data (such as nonces and initialization data) before the encryption or decryption process starts. It is considered a crucial performance metric, especially for sponge-based algorithms, as their performance is highly sensitive to initialization overhead. Figure 3 illustrates TinyJambu's remarkable efficiency with 38.12 μ s. In contrast, AES incurs substantial overhead, taking 486 μ s, by requiring re-initialization for each new data block.

3) *Time Required to Encrypt One Byte*: To investigate the second RQ, it is essential to perform a thorough analysis of one-byte encryption time to conduct a detailed analysis of the time required to encrypt a single byte. Figure 4 compares LWC algorithms based on the time required to encrypt one byte. The Ascon algorithm provides minimal time for encrypting one byte, 50 μ s. TinyJambu ranks second, with a time of 55.8 μ s.

4) *Throughput*: The throughput and CPU cycles per byte analysis directly address the third RQ, highlighting the suitability of LWC algorithms for IIoT environments. Figure 5 presents the throughput measurements for algorithms with a plaintext data size of 5 kB, ensuring that the data size exceeds the minimum block size of all algorithms. Regarding overall throughput measurement,

the Ascon, Schwaemm, and TinyJambu algorithms generally exhibit higher throughput than AES GCM.

5) *CPU cycles per Byte*: Figure 6 illustrates the comparison of algorithms based on the number of CPU cycles required to encrypt a single byte. This statistic measures the computational efficiency of each algorithm, indicating the computational workload associated with encrypting a single byte. Fewer CPU cycles per byte indicate greater efficiency and enhanced appropriateness for resource-limited settings. We observe that the Ascon and Schwaemm algorithms exhibit the lowest CPU cycles. Additionally, across different variants of the Ascon algorithm, the effort needed to encrypt a single byte remains consistently low, further highlighting its efficiency.

6) *Memory Usage*: Figure 7 illustrates each algorithm's memory usage, which gives insight into how efficiently it uses memory resources throughout cryptographic operations. Additionally, the analysis shows that TinyJambu has one of the lowest memory footprints among all evaluated algorithms. This metric shows that all 10 LWCs consume significantly less memory than AES.

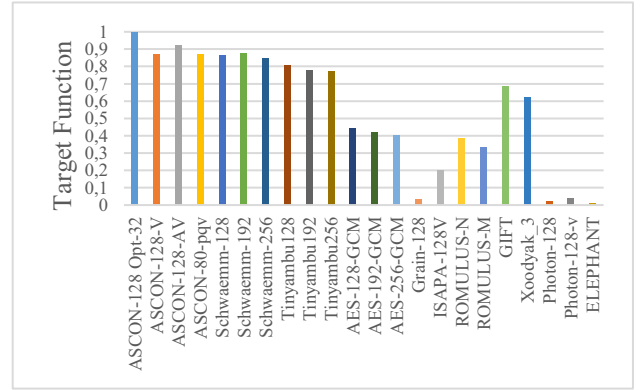


Figure 2. The target function of evaluating LWC

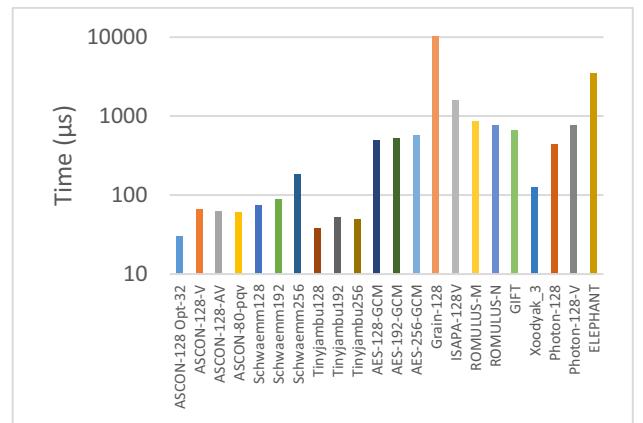


Figure 3. Total time of the initialization process

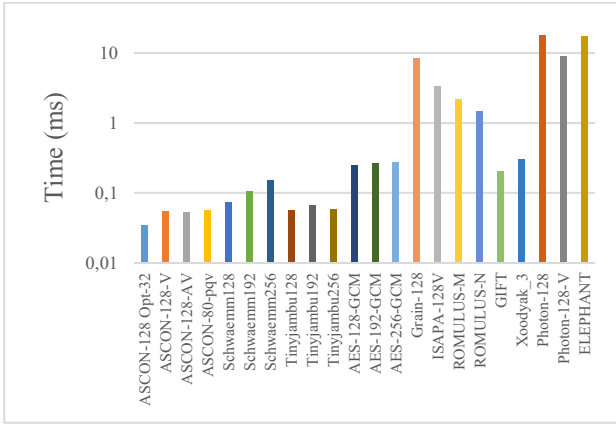


Figure 4. Time required to encrypt one Byte (Data size 5 Kbytes)

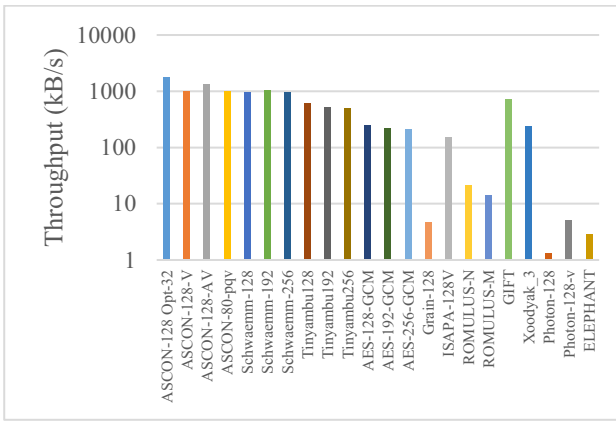


Figure 5. Throughput of LWC algorithms with data size 5kByte

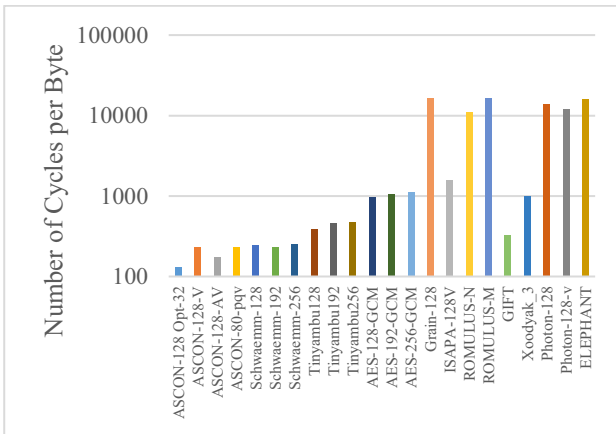


Figure 6. Number of Cycles per Byte with 5kByte Data

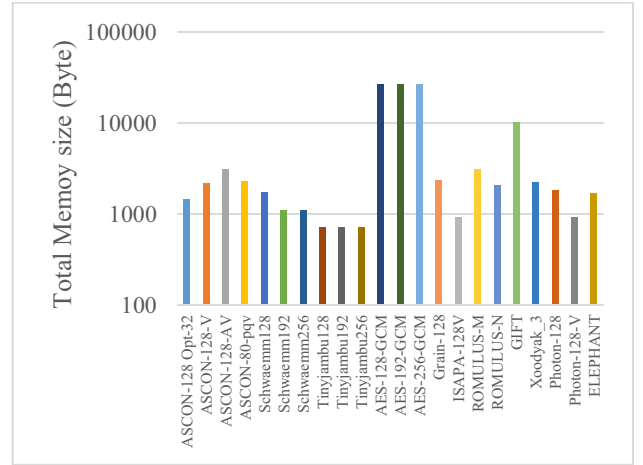


Figure 7. Memory usage comparison of LWC algorithms

7) *CPU Utilization*: Figure 8 demonstrates that LWC algorithms, like Photon-Beetle and Grain, benefit from multi-threaded architectures for improved performance. In particular, the Grain algorithm efficiently generates the keystream by mixing with plaintext or ciphertext using a minimum of two threads.

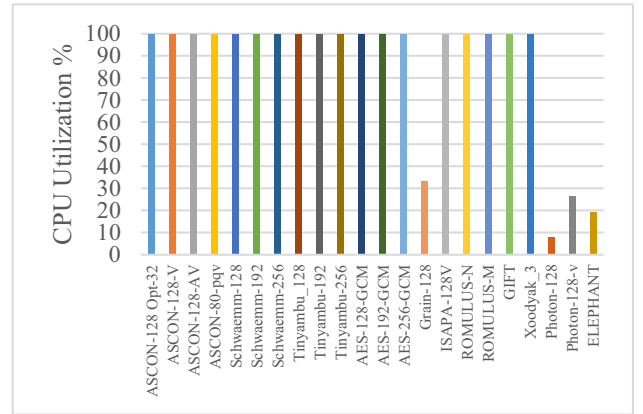


Figure 8. CPU utilization of LWC and AES

V. DISCUSSIONS

This study provides insightful information regarding LWC algorithms operating on IIoT devices. To meet the requirements of the IIoT, the target function sets priorities for key performance indicators. With a high target function score of 0.91, the target function analysis considers ASCON-128 variants as the premier choice for LWC algorithms. It uses 234 CPU cycles per byte, which is significantly lower than AES-128's 959 cycles per byte, making it faster to encrypt a single byte with a rapid initialization time of just 65 μ s, significantly quicker than AES-128's 486 μ s. Schwaemm ranks second with a target function evaluation score of 0.86 and achieves outstanding performance at 246 cycles per byte. Thanks to its minimal memory footprint and overall balanced performance, TinyJambu performs exceptionally well in environments with limited memory. These combined advantages make

ASCON-128 particularly well-suited for resource-constrained devices, while TinyJambu excels in memory-constrained environments.

The initialization process time has emerged as a crucial performance metric, especially for sponge-based algorithms, where throughput is directly affected. TinyJambu demonstrates high efficiency with a latency of 38.12 μ s. In contrast, AES introduces a significant overhead, taking 486 μ s, by requiring re-initialization for each new data block. Data block size significantly affects throughput and memory usage. Larger block sizes consistently deliver higher throughput, as evidenced by ASCON-VA achieving 1.341 MB/s compared to ASCON-V's 0.997 MB/s, despite the increased number of rounds from 30 to 32.

VI. CONCLUSIONS

LWC algorithms will occupy a larger space in future applications and protocols due to their versatility across applications, extending beyond use in constrained devices. This study demonstrates that sponge-based algorithms like ASCON and TinyJambu can achieve higher throughput as encrypted data scales since the initialization process remains constant. This makes them suitable for multimedia applications where large data volumes are involved, as throughput remains stable regardless of payload size.

The results demonstrate that increasing the block data size can improve the performance of LWC algorithms such as ASCON 128 AV. In contrast, the ASCON-pqv80 results indicate that increasing key sizes may not always enhance performance. The findings suggest that ASCON can provide high throughput with different data sizes and reliable performance, but also uses significant memory.

CPU utilization results revealed that specific LWC algorithms, such as PHOTON-Beetle and Grain, exhibit higher CPU usage in software implementations. These algorithms are tailored to specific applications; for instance, PHOTON is optimized for hardware implementation. This variety in LWC's performance reflects its goal of providing high performance across various operating environments.

Future research should prioritize two important directions in LWC: first, benchmarking LWC algorithms on hardware platforms, mainly using FPGAs, where algorithm design constraints may substantially alter the performance of the algorithms. Second, a systematic integration of LWC with standard framework security protocols such as IPSec, TLS, and S_MQTT is an undiscovered but essential area. A successful integration of these technologies would confirm LWC's dual capability to maintain robust security while preserving resource efficiency, ultimately proving its feasibility for use in various constrained contexts, from network infrastructure components to embedded IoT devices.

VII. ACKNOWLEDGMENT

The authors gratefully acknowledge funding from the German Academic Exchange Service (DAAD) in the

framework of the project "Expanded Research, Development & Teaching on Internet of Things for Sustainable, Secure, and Safe Communities (ERTISSS)", project #57727462, 2024.

VIII. DECLARATION ON GENERATIVE AI

During the preparation of this paper, the authors used Grammarly to check spelling and grammar. The authors subsequently reviewed and revised the content and take responsibility for its final form and content.

IX. REFERENCES

- [1] L. S. Vailshery, "Number of IoT connections worldwide 2022-2033," 11 9 2024. [Online]. Available: <https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide>. [Accessed 20 12 2024].
- [2] GSMA, "Safety, privacy and security across the mobile ecosystem," GSMA, London, 2022.
- [3] M. Skuballa, A. Walz, H. Bühler and A. Sikora, "Cryptographic Protection of Cyclic Real-Time Communication in Ethernet-Based Fieldbuses: How Much Hardware is Required?," 2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA), pp. 1-7, 2021.
- [4] H. M. Alshahrani, "CoLL-IoT: A Collaborative Intruder Detection System for Internet of Things Devices," *Electronics*, vol. 10, no. 7, pp. 1-19, 2021.
- [5] X. Wei, M. El-Hadedy, S. Mosanu, Z. Zhu, W.-M. Hwu and X. Guo, "RECO-HCON: A High-Throughput Reconfigurable Compact ASCON Processor for Trusted IoT," 2022 IEEE 35th International System-on-Chip Conference (SOCC), pp. 1-6, 2022.
- [6] H. Bühler, A. Walz and A. Sikora, "Benchmarking of Symmetric Cryptographic Algorithms on a Deeply Embedded System," *IFAC-PapersOnLine*, vol. 55, no. 4, pp. 266-271, 2022.
- [7] W. J. Buchanan, S. Li and R. Asif, "Lightweight cryptography methods," *Journal of Cyber Security Technology*, vol. 1, no. 3-4, pp. 187-201, 10 2017.
- [8] C. Dobraunig, M. Eichlseder, F. Mendel and M. Schläffer, "Ascon v1.2: Lightweight Authenticated Encryption and Hashing," *Journal of Cryptology*, vol. 34, no. 3, p. 33, 2021.
- [9] S. Banik, A. Chakraborti, T. Iwata, K. Minematsu, M. Nandi, T. Peyrin, Y. Sasaki, S. M. Sim and Y. Todo, "GIFT-COFB: NIST LWC Second-round Candidate Status Update," Submitted to NIST Lightweight Cryptography,, 2020.
- [10] C. Dobraunig, M. Eichlseder, S. Mangard, F. Mendel, B. Mennink, R. Primas and T. Unterluggauer, "ISAP v2.0. Submission to NIST," National Institute of Standards and Technology, 2021.
- [11] A. Chakraborti, N. Datta, M. Nandi and K. Yasuda, "Beetle Family of Lightweight and Secure Authenticated Encryption Ciphers," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 2, pp. 218-241, 5 2018.
- [12] C. Beierle, A. Biryukov, L. Cardoso dos Santos, J. Großschädl, A. Moradi, L. Perrin, A. Rezaei Shahmirzadi, A. Udovenko, V. Velichkov and Q. Wang, "Schwaemm and Esch: Lightweight Authenticated Encryption and Hashing using the Sparkle Permutation Family Corresponding submitter," NIST, 2021.
- [13] S. Kumar, J. Haj-Yahya, M. Khairallah, M. A. Elmoher and A. Chattopadhyay, "A Comprehensive Performance Analysis of Hardware Implementations of CAESAR Candidates," *Cryptology ePrint Archive*, 2017.
- [14] G. Bertoni, J. Daemen, M. Peeters and G. Van Assche, "Duplexing the Sponge: Single-Pass Authenticated Encryption and Other Applications," *Selected Areas in Cryptography*, pp. 320-337, 2012.
- [15] C. Dobraunig, M. Eichlseder, F. Mendel and M. Schläffer, "Ascon v1.2. Submission to NIST," NIST, 2021.
- [16] H. Wu and T. Huang, "JAMBU Lightweight Authenticated Encryption Mode and AES-JAMBU," NIST Lightweight Cryptography Workshop, 2015.
- [17] G. M. Bertoni, J. Daemen and M. Peeters, "Keccak sponge function family main document," NIST (Round 2), vol. 3, 1 2009.

- [18] M. Hell, T. Johansson, L. University, S. A. Maximov, E. Ab, W. Meier, J. Sönnerup and H. Yoshida, "Grain-128AEADv2-A lightweight AEAD stream cipher Cover sheet Corresponding submitter: Backup point of contact," NIST, Tokyo, 2020
- [19] R. S. Mahantesh and S. Mohapatra, "Design of Secured Block Ciphers PRESENT and HIGHT Algorithms and its FPGA Implementation," 2018 Second International Conference on Intelligent Computing and Control Systems (ICICCS), pp. 1113-1118, 2018.
- [20] I. Elsadek, S. Aftabjahani, D. Gardner, E. MacLean, J. R. Wallrabenstein and E. Y. Tawfik, "Hardware and Energy Efficiency Evaluation of NIST Lightweight Cryptography Standardization Finalists," 2022 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 133-137, 2022.
- [21] T. Kitahara, R. Hira, Y. Hara-Azumi, D. Miyahara, Y. Li and K. Sakiyama, "Optimized Software Implementations of Ascon, Grain-128AEAD, and TinyJambu on ARM Cortex-M0," Proceedings - 2022 10th International Symposium on Computing and Networking Workshops, CANDARW 2022, pp. 316-322, 2022.
- [22] C. Rahul, N. Kousarr, T. A. Yadav, P. Keerthi, S. Hariharan and V. Kukreja, "Analysis of Resource Utilization in Lightweight Cryptographic Algorithms," 2024 International Conference on Cognitive Robotics and Intelligent Systems, ICC - ROBINS 2024, pp. 884-889, 2024.
- [23] M. El-hajj, H. Mousawi and A. Fadlallah, "Analysis of Lightweight Cryptographic Algorithms on IoT Hardware Platform," Future Internet, vol. 15, no. 2, 2 2023.